

```
print('')

# Query 2 on the relational table will display the sum
of salaries & count of employees by grouping department-
wise

res2 = pd.read_sql_query('SELECT dept,sum(salary) as
salary_sum,count(*) empcount FROM data_table group by
dept', engine)
print('Result 2', res2)
```

#### Output

Result 1 will display the entire table Result 2

|   | dept | salary_sum | empcount |
|---|------|------------|----------|
| 0 | CSE  | 105586     | 3        |
| 1 | ECE  | 51600      | 2        |
| 2 | IT   | 63500      | 2        |
| 3 | ME   | 32100      | 1        |

## Insertion and deletion on the database

Inserting and updating rows into an existing SQLite database table, apart from sending queries, is probably the most common database operation. Sqlite3 has a straightforward way to inject dynamic values without relying on string formatting.

Any '?' value in the query will be replaced by a value in values. The first '?' will be replaced by the first item in values, the second by the second item, and so on. This works for any type of query. This creates an SQLite parametrised query, which avoids SQL injection issues.

Finally, we can delete the rows in a database using the *execute* method, as shown below:

```
from sqlalchemy import create_engine
from pandas.io import sql

import pandas as pd

data = pd.read_csv('C:/Users/Neethu/inputdata.csv')
engine = create_engine('sqlite:///memory:')

# Store the Data in a relational table
data.to_sql('data_table', engine)

# Insert another row
sql.execute('INSERT INTO data_
table VALUES(?,?,?,?,?,?)', engine,
params=[('id', 9, 'Rose', 34320, '20-03-18', 'IT')])
#Update the salary for IT dept
sql.execute('UPDATE data_table set salary=(?) where
dept=(?) ', engine, params=[('89000', 'ECE')])
#Delete row where name=Neha
sql.execute('Delete from data_table where name = (?) ',
```

```
engine, params=[('Neha')])

# Read from the relational table
res = pd.read_sql_query('SELECT ID,Dept,Name,Salary,start_
date FROM data_table', engine)
print(res)
```

For the output, see Figure 2.

|   | id | dept | name  | salary | start_date |
|---|----|------|-------|--------|------------|
| 0 | 1  | CSE  | John  | 34000  | 05-08-17   |
| 1 | 2  | ECE  | Manu  | 29600  | 23-09-18   |
| 2 | 3  | IT   | Isha  | 32500  | 05-12-18   |
| 3 | 4  | CSE  | Ryan  | 35678  | 01-05-19   |
| 4 | 5  | IT   | Pooja | 31100  | 03-05-19   |
| 5 | 6  | ECE  | Sam   | 31000  | 30-07-17   |
| 6 | 7  | CSE  | Sanju | 26000  | 27-06-18   |
| 7 | 8  | IT   | Rose  | 35908  | 20-03-18   |

Figure 2: The table, after the insert, update and delete queries

To recapitulate, Python is a great language for doing data analysis, primarily because of the fantastic ecosystem of data-centric Python packages, numerous libraries and built-in features. This makes it easy to tackle the needs of data science. Pandas is a powerful data analysis tool kit, which is intended to be a high-level building block for actual data analysis in Python. SQLite is the most widely deployed SQL database engine in the world, which can be integrated with Python using a Python module called Sqlite3. Compared to writing the traditional raw SQL statements using Sqlite3, SQLAlchemy's code is more object-oriented, and easier to read and maintain. 

## References

- [1] [https://www.tutorialspoint.com/python/python\\_data\\_operations.htm](https://www.tutorialspoint.com/python/python_data_operations.htm)
- [2] <https://opentechschoo.github.io/python-data-intro/core/charts.html>
- [3] <https://realpython.com/python-json/>
- [4] [https://www.tutorialspoint.com/python/python\\_relational\\_databases.htm](https://www.tutorialspoint.com/python/python_relational_databases.htm)

## By: Neethu C. Sekar

The author is an open source enthusiast, currently working as an assistant professor in the Department of Computer Science, AmalJyothi College of Engineering, Kerala.